

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по индивидуальному домашнему заданию
по дисциплине «Распределенные алгоритмы»

Тема: Реализация алгоритма на языке Promela и верификация в Spin

Студенты гр. 3343

Бондаренко Ф.А.

Пименов П.В.

Преподаватель

Шошмина И.В.

Санкт-Петербург

2026

Задание.

- Разработайте параллельный алгоритм умножения двух матриц $n \times n$ для P процессов так, чтобы параллельно вычислялись строки результирующей матрицы. Исходные матрицы загружены на процессы заранее.
- Сформулировать свойства алгоритма на языке LTL.
- Обмен данными, если он необходим, осуществляется с помощью каналов.
- По завершении умножения 0-й процесс выводит результат.

Выполнение работы.

1. Архитектура и распределение данных

Для моделирования алгоритма в среде Spin массивы реализованы в виде одномерных структур $A[N*N]$, $B[N*N]$ и $C[N*N]$, имитирующих двумерные матрицы размером $N*N$, где доступ к элементу (row, col) осуществляется по формуле $row * N + col$. Исходные значения матриц загружаются предварительно в блоке `init`.

Используется парадигма параллелизма по данным (Data Parallelism). Распределение работы между P процессами (worker) осуществляется циклически по строкам (Cyclic Distribution). Каждый процесс получает идентификатор `id` (от 0 до $P-1$) и обрабатывает строки с индексами `row`, для которых выполняется условие $row \bmod P == id$.

2. Синхронизация и обмен данными

В соответствии с заданием, для обмена данными используются каналы сообщений.

Для передачи вычисленных элементов результирующей матрицы C от рабочих процессов к нулевому процессу (агрегатору) объявлен глобальный канал:

```
chan row_computed = [N * N] of { byte, byte, int };
```

Процессы-воркеры ($id \neq 0$): Вычислив скалярное произведение для текущего элемента, отправляют в канал кортеж {номер_строки, номер_столбца, значение}.

Нулевой процесс ($id = 0$): Вычисляет свою долю строк матрицы, записывая их напрямую в результирующий массив C . После завершения своих вычислений агрегатор высчитывает по формуле точное количество элементов (`expected_elements`), которое должны вернуть остальные воркеры. Затем нулевой процесс блокируется на чтении из канала, ожидая недостающие элементы, сохраняет их в матрицу C и выводит готовый результат на экран.

3. Формирование и верификация LTL-свойств

Для доказательства корректности работы алгоритма и отсутствия тупиковых ситуаций (дедлоков) были сформулированы свойства живости (liveness) и свойство безопасности (safety).

В код введен глобальный флаг `bool done = false`, который переключается в `true` нулевым процессом строго после того, как все вычисления завершены, данные получены и результат выведен.

Свойства сформулированы на языке LTL следующим образом:

```
ltl liveness { <> (done == true) }  
ltl safety { (done -> (C[0] == 43 && C[1] == 109 && C[2] ==  
45 && C[3] == 31 && C[4] == 16 && C[5] == 11 && C[6] == 73 && C[7]  
== 51 && C[8] == 31)) }
```

Формула `liveness` означает: «в любом пути выполнения системы рано или поздно наступит состояние, в котором флаг `done` станет истинным». Формула `safety` означает: «как только вывод завершен, значения результирующей матрицы соответствуют ожидаемому результату на тестовых данных».

4. Результаты полной верификации в Spin:

При запуске анализатора с проверкой циклов принятия (acceptance cycles) был получен следующий результат:

```
nreached in claim liveness  
  _spin_nvr.tmp:6, state 6, "-end-"  
  (1 of 6 states)  
unreached in claim safety  
  _spin_nvr.tmp:15, state 8, "-end-"  
  (1 of 8 states)
```

Статус `unreached` ("не достигнуто") для финального состояния внутри автомата-ловушки (`never claim`) однозначно доказывает, что отрицание наших LTL-свойств невозможно. Модель гарантированно завершается успехом, вычисляет матрицу и не имеет тупиковых блокировок или состояний гонки (`race conditions`). Данные через каналы передаются без потерь.

5. Сравнение с альтернативным подходом

В ходе работы был проведен анализ решения другой группы, которая реализовывала передачу данных механизмом общей (разделяемой) памяти и отслеживанием статуса выполнения через глобальные переменные.

Ключевые отличия:

- **Архитектура синхронизации:** В нашем решении синхронизация потоков естественным образом вытекает из блокирующего чтения из канала (`row_computed ? r, c, val`). Агрегатор переходит в режим ожидания (`suspend`) до появления данных в буфере. В решении коллег рабочие процессы осуществляют запись непосредственно в глобальный массив `C`, а синхронизация осуществляется через инкремент глобального атомарного счетчика завершенных задач (`atomic { finished = finished + 1 }`) и последующее "охранное" условие-барьер (`finished == P`), которое блокирует нулевой процесс до момента завершения всех воркеров.
- **Производительность:** Работа с общей памятью генерирует менее объемное пространство состояний в `Spin` (меньший размер вектора состояний (`State-vector`) и меньше `transitions`), так как симулятору не требуется моделировать и сохранять промежуточные состояния буферов каналов при отправке и чтении каждого отдельного элемента.
- **Надежность:** Концепция передачи данных через разделяемую память потенциально небезопасна из-за высокого риска состояний гонки (`data races`). В алгоритме коллег гонок удалось избежать благодаря строгому непересекающемуся разделению по строкам (отсутствует общая перезапись). Однако канальная архитектура (наш вариант) концептуально гораздо

надежнее: она снижает связность компонентов и гарантирует потокобезопасность на архитектурном уровне, так как модификацией итоговой структуры *C* занимается исключительно монопольный процесс-агрегатор (0-й воркер).

Выводы.

В ходе выполнения лабораторной работы был успешно спроектирован, реализован на языке *Promela* и верифицирован в среде *Spin* параллельный алгоритм перемножения квадратных матриц. Использование каналов передачи сообщений (*chan*) показало себя как надежный инструмент синхронизации: нулевой процесс выступал в роли точки синхронизации и успешно агрегировал независимые вычисления рабочих потоков. С помощью строгой проверки LTL-свойств (*liveness*, *safety*) было доказано, что созданная математическая модель алгоритма свободна от взаимных блокировок (*deadlocks*), а полное вычисление матрицы и её вывод гарантированно завершаются при любых возможных вариантах планирования (*scheduling*) потоков. По результатам обмена опытом с другой группой было установлено, что хотя подход с разделяемой памятью и барьерной синхронизацией генерирует меньшую нагрузку на *State-space* верификатора, выбранная в нашей работе парадигма передачи сообщений (*Message Passing*) позволяет писать более масштабируемый и надежный с точки зрения *data races* распределенный код. Дополнительно разработанный скрипт автоматического тестирования на *Python* подтвердил арифметическую корректность возвращаемых алгоритмом данных на матрицах и процессах различной размерности. Полный исходный код программы см. в Приложении А.

ПРИЛОЖЕНИЕ А

Файл main.pml:

```
// Размер квадратных матриц N*N
#define N 3
// Количество параллельных процессов
#define P 2

// Одномерные массивы для хранения двумерных матриц N*N
int A[N * N];
int B[N * N];
int C[N * N];

// Флаг завершения всех вычислений (для верификации LTL)
bool done = false;

// Канал для передачи данных от рабочих процессов к нулевому.
// Передается: { номер строки, номер столбца, вычисленное значение }
chan row_computed = [N * N] of { byte,byte,int };

// Функция рабочего процесса (воркера)
proctype worker(byte id) {
// Процесс берет строки с шагом P, начиная со строки = id
    byte row = id;
    byte col,k;
    int sum;

// Цикл по строкам, закрепленным за данным процессом
    do
        :: row < N ->
            col = 0;
// Цикл по столбцам матрицы B
            do
                :: col < N ->
                    sum = 0;
                    k = 0;
// Вычисление скалярного произведения строки A и столбца B
                    do
                        :: k < N ->
                            sum = sum + A[row * N + k] * B[k * N + col];
                            k = k + 1;
                        :: k == N -> break;
                    od;

// Если это нулевой процесс, он сразу пишет в итоговую матрицу
                    if
                        :: id == 0 ->
                            C[row * N + col] = sum;
// Остальные процессы отправляют результат через канал нулевому процессу
                        :: id != 0 ->
                            row_computed!row,col,sum;
                    fi;
                    col = col + 1;
                :: col == N -> break;
            od;
// Переход к следующей строке, закрепленной за этим процессом
            row = row + P;
            :: row >= N -> break;
        od;

// Только нулевой процесс занимается агрегацией и выводом
        if
            :: id == 0 ->
```

```

        int i,j;
        int expected_elements = 0;
        i = 0;
// Подсчитываем, сколько элементов мы должны получить из канала (сколько элементов
вычислили другие процессы)
        do
            :: i < N ->
                if
                    :: (i % P) != 0 -> expected_elements = expected_elements + N;
                    :: else -> skip;
                fi;
                i = i + 1;
            :: i == N -> break;
        od;

        int received = 0;
        byte r,c;
        int val;
// Цикл ожидания и чтения вычисленных элементов из канала
        do
            :: received < expected_elements ->
                row_computed?r,c,val;
// Запись полученного элемента в результирующую матрицу
                C[r * N + c] = val;
                received = received + 1;
            :: received == expected_elements -> break;
        od;

// Вывод итоговой матрицы C
        printf("Result Matrix C:\n");
        i = 0;
        do
            :: i < N ->
                j = 0;
                printf("[");
                do
                    :: j < N ->
                        if
                            :: j < N - 1 -> printf("%d,",C[i * N + j]);
                            :: j == N - 1 -> printf("%d",C[i * N + j]);
                        fi;
                        j = j + 1;
                    :: j == N -> break;
                od;
                printf("]\n");
                i = i + 1;
            :: i == N -> break;
        od;

// Устанавливаем флаг завершения работы
        done = true;
        :: id != 0 -> skip;
        fi;
    }

init {
// Тестовые данные
    A[0] = 5;A[1] = 3;A[2] = 7;
    A[3] = 1;A[4] = 4;A[5] = 0;
    A[6] = 3;A[7] = 9;A[8] = 1;

    B[0] = 3;B[1] = 8;B[2] = 7;
    B[3] = 7;B[4] = 2;B[5] = 1;

```



```

        B[6] = 1; B[7] = 9; B[8] = 1;
// Ожидаемый результат
// [43,109,45]
// [31,16,11]
// [73,51,31]

// Атомарный запуск P рабочих процессов
int proc_id = 0;
atomic {
    do
        :: proc_id < P ->
            run worker(proc_id);
            proc_id = proc_id + 1;
        :: proc_id == P -> break;
    od;
}

// Свойство живости
ltl liveness { <> (done == true) }

// Свойство безопасности
ltl safety { (done -> (C[0] == 43 && C[1] == 109 && C[2] == 45 &&
    C[3] == 31 && C[4] == 16 && C[5] == 11 &&
    C[6] == 73 && C[7] == 51 && C[8] == 31)) }

```